CLI Usage

CLI Usage
- 0. Enter Docker
- 1. Using CLI for Prediction
- 2. CLI Usage Syntax
- 3. Image Prediction
  - Effect Preview
- 4. Video Prediction
- 5. Real-time Prediction
- References

0. Enter Docker

JetsonNano version needs to enter docker for use, run the following command

```
~/run_yolo_docker.sh
```

All subsequent commands are implemented in docker

1. Using CLI for Prediction

CLI (Command-Line Interface) is a way for users to interact with computers or software programs, where users run programs or execute tasks by typing text commands instead of clicking icons or buttons through a graphical user interface (GUI).

2. CLI Usage Syntax

```
yolo TASK MODE ARGS

Where   TASK (optional) is one of [detect, segment, classify, pose, obb]
        MODE (required) is one of [train, val, predict, export, track,
benchmark]
        ARGS (optional) are any number of custom 'arg=value' pairs like
'imgsz=320' that override defaults.
```

3. Image Prediction

Use yolo26n.pt to predict images that come with the ultralytics26 project: If the system cannot find the corresponding model file in the directory where the command is run, it will automatically download (if download fails, you can copy the model in yourself)

Enter the project folder:

```
cd /ultralytics/
```

Use yolo26n.pt to detect images in the target folder and output results:

```
yolo predict model=yolo26n.pt source='/root/yolo26_data/bus.jpg'
```

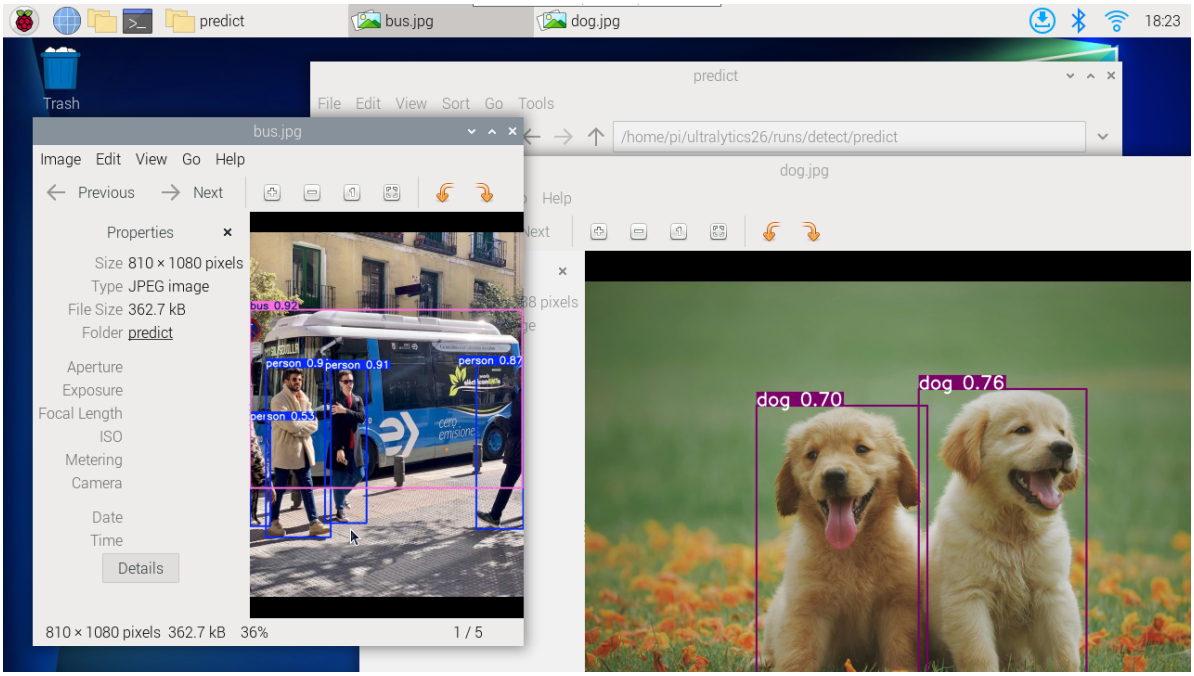
```
root@jetson-yahboom:/ultralytics# yolo predict model=yolo26n.pt source='/root/yolo26_data/bus.jpg'
Ultralytics 8.4.24 🚀 Python-3.8.0 torch-1.11.0a0+gitbc2c6ed CUDA:0 (NVIDIA Tegra X1, 3964MiB)
/ultralytics/ultralytics/nn/modules/head.py:246: UserWarning: __floordiv__ is deprecated, and its behavior will change in a future vers
ion of pytorch. It currently rounds toward 0 (like the 'trunc' function NOT 'floor'). This results in incorrect rounding for negative v
alues. To keep the current behavior, use torch.div(a, b, rounding_mode='trunc'), or for actual floor division, use torch.div(a, b, roun
ding_mode='floor').
  idx = ori_index[torch.arange(batch_size)[..., None], index // nc]  # original index
YOLO26n summary (fused): 122 layers, 2,408,932 parameters, 0 gradients, 5.4 GFLOPs

image 1/1 /root/yolo26_data/bus.jpg: 640x480 4 persons, 1 bus, 130.9ms
Speed: 14.6ms preprocess, 130.9ms inference, 2.4ms postprocess per image at shape (1, 3, 640, 480)
Results saved to /ultralytics/runs/detect/predict2
💡 Learn more at https://docs.ultralytics.com/modes/predict
```

Effect Preview

yolo recognition output image location: /ultralytics/runs/detect/predict2

Each time it runs, the predict under this folder automatically increments by 1



4. Video Prediction

> JetsonNano may experience insufficient memory, in this case please try shorter or smaller videos

Use yolo26n.pt to predict videos in the ultralytics project (not videos that come with ultralytics): If the system cannot find the corresponding model file in the directory where the command is run, it will automatically download (if download fails, you can copy the model in yourself)

Enter the project folder:

```
cd /ultralytics
```

Use yolo26n.pt to detect target videos and output results:

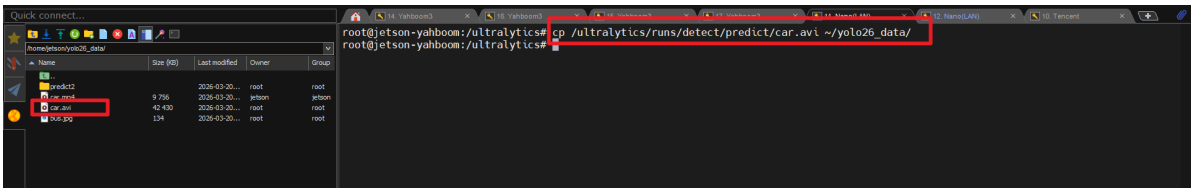
```
yolo predict model=yolo26n.pt source='/root/yolo26_data/car.mp4'
```

```
root@jetson-yahboom:/ultralytics# yolo predict model=yolo26n.pt source='/root/yolo26_data/car.mp4'
Ultralytics 8.4.24 🚀 Python-3.8.0 torch-1.11.0a0+gitbc2c6ed CUDA:0 (NVIDIA Tegra X1, 3964MiB)
/ultralytics/ultralytics/nn/modules/head.py:246: UserWarning: __floordiv__ is deprecated, and its behavior will change in a future vers
ion of pytorch. It currently rounds toward 0 (like the 'trunc' function NOT 'floor'). This results in incorrect rounding for negative v
alues. To keep the current behavior, use torch.div(a, b, rounding_mode='trunc'), or for actual floor division, use torch.div(a, b, roun
ding_mode='floor').
  idx = ori_index[torch.arange(batch_size)[..., None], index // nc]  # original index
YOLO26n summary (fused): 122 layers, 2,408,932 parameters, 0 gradients, 5.4 GFLOPs

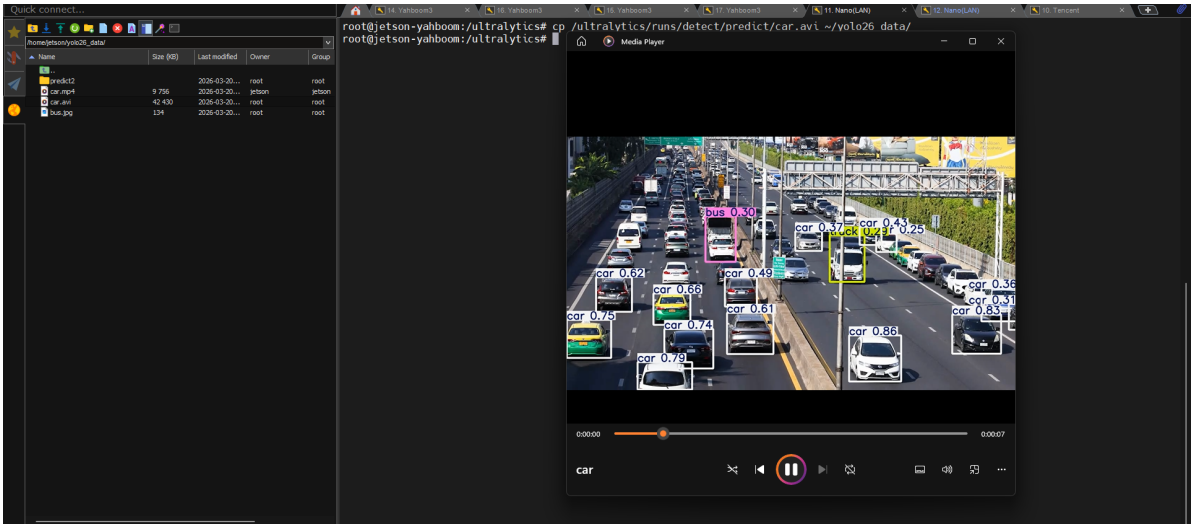
video 1/1 (frame 1/180) /root/yolo26_data/car.mp4: 384x640 12 cars, 1 bus, 110.8ms
video 1/1 (frame 2/180) /root/yolo26_data/car.mp4: 384x640 15 cars, 1 bus, 113.8ms
video 1/1 (frame 3/180) /root/yolo26_data/car.mp4: 384x640 13 cars, 1 bus, 107.3ms
video 1/1 (frame 4/180) /root/yolo26_data/car.mp4: 384x640 14 cars, 1 bus, 107.7ms
video 1/1 (frame 5/180) /root/yolo26_data/car.mp4: 384x640 15 cars, 1 bus, 107.2ms
video 1/1 (frame 6/180) /root/yolo26_data/car.mp4: 384x640 13 cars, 108.6ms
video 1/1 (frame 7/180) /root/yolo26_data/car.mp4: 384x640 14 cars, 108.8ms
video 1/1 (frame 8/180) /root/yolo26_data/car.mp4: 384x640 13 cars, 114.4ms
video 1/1 (frame 9/180) /root/yolo26_data/car.mp4: 384x640 17 cars, 119.5ms
video 1/1 (frame 10/180) /root/yolo26_data/car.mp4: 384x640 17 cars, 110.7ms
video 1/1 (frame 11/180) /root/yolo26_data/car.mp4: 384x640 15 cars, 115.2ms
video 1/1 (frame 12/180) /root/yolo26_data/car.mp4: 384x640 16 cars, 111.4ms
video 1/1 (frame 13/180) /root/yolo26_data/car.mp4: 384x640 14 cars, 108.5ms
video 1/1 (frame 14/180) /root/yolo26_data/car.mp4: 384x640 17 cars, 112.9ms
video 1/1 (frame 15/180) /root/yolo26_data/car.mp4: 384x640 15 cars, 115.4ms
video 1/1 (frame 16/180) /root/yolo26_data/car.mp4: 384x640 17 cars, 132.4ms
video 1/1 (frame 17/180) /root/yolo26_data/car.mp4: 384x640 16 cars, 1 traffic light, 122.0ms
video 1/1 (frame 18/180) /root/yolo26_data/car.mp4: 384x640 14 cars, 1 traffic light, 114.5ms
video 1/1 (frame 19/180) /root/yolo26_data/car.mp4: 384x640 15 cars, 136.6ms
video 1/1 (frame 20/180) /root/yolo26_data/car.mp4: 384x640 15 cars, 1 bus, 119.1ms
video 1/1 (frame 21/180) /root/yolo26_data/car.mp4: 384x640 14 cars, 1 bus, 110.1ms
video 1/1 (frame 22/180) /root/yolo26_data/car.mp4: 384x640 13 cars, 1 bus, 108.2ms
video 1/1 (frame 23/180) /root/yolo26_data/car.mp4: 384x640 14 cars, 1 truck, 107.0ms
video 1/1 (frame 24/180) /root/yolo26_data/car.mp4: 384x640 14 cars, 1 bus, 1 truck, 105.1ms
video 1/1 (frame 25/180) /root/yolo26_data/car.mp4: 384x640 14 cars, 128.1ms
video 1/1 (frame 26/180) /root/yolo26_data/car.mp4: 384x640 16 cars, 1 truck, 114.2ms
video 1/1 (frame 27/180) /root/yolo26_data/car.mp4: 384x640 16 cars, 1 bus, 1 truck, 108.3ms
video 1/1 (frame 28/180) /root/yolo26_data/car.mp4: 384x640 14 cars, 1 truck, 107.6ms
video 1/1 (frame 29/180) /root/yolo26_data/car.mp4: 384x640 15 cars, 1 bus, 107.9ms
```

```
video 1/1 (frame 166/180) /root/yolo26_data/car.mp4: 384x640 15 cars, 4 trucks, 102.1ms
video 1/1 (frame 167/180) /root/yolo26_data/car.mp4: 384x640 15 cars, 2 trucks, 106.5ms
video 1/1 (frame 168/180) /root/yolo26_data/car.mp4: 384x640 16 cars, 2 trucks, 106.6ms
video 1/1 (frame 169/180) /root/yolo26_data/car.mp4: 384x640 16 cars, 3 trucks, 106.7ms
video 1/1 (frame 170/180) /root/yolo26_data/car.mp4: 384x640 15 cars, 2 trucks, 106.6ms
video 1/1 (frame 171/180) /root/yolo26_data/car.mp4: 384x640 13 cars, 2 trucks, 106.5ms
video 1/1 (frame 172/180) /root/yolo26_data/car.mp4: 384x640 14 cars, 2 trucks, 106.5ms
video 1/1 (frame 173/180) /root/yolo26_data/car.mp4: 384x640 13 cars, 2 trucks, 106.5ms
video 1/1 (frame 174/180) /root/yolo26_data/car.mp4: 384x640 13 cars, 2 trucks, 106.4ms
video 1/1 (frame 175/180) /root/yolo26_data/car.mp4: 384x640 15 cars, 4 trucks, 106.5ms
video 1/1 (frame 176/180) /root/yolo26_data/car.mp4: 384x640 14 cars, 2 trucks, 106.5ms
video 1/1 (frame 177/180) /root/yolo26_data/car.mp4: 384x640 17 cars, 2 trucks, 106.5ms
video 1/1 (frame 178/180) /root/yolo26_data/car.mp4: 384x640 16 cars, 106.4ms
video 1/1 (frame 179/180) /root/yolo26_data/car.mp4: 384x640 17 cars, 1 truck, 106.4ms
video 1/1 (frame 180/180) /root/yolo26_data/car.mp4: 384x640 18 cars, 2 trucks, 106.3ms
Speed: 7.0ms preprocess, 106.2ms inference, 3.0ms postprocess per image at shape (1, 3, 384, 640)
Results saved to /ultralytics/runs/detect/predict
💡 Learn more at https://docs.ultralytics.com/modes/predict
root@jetson-yahboom:/ultralytics#
```

We copy this generated file to the shared directory

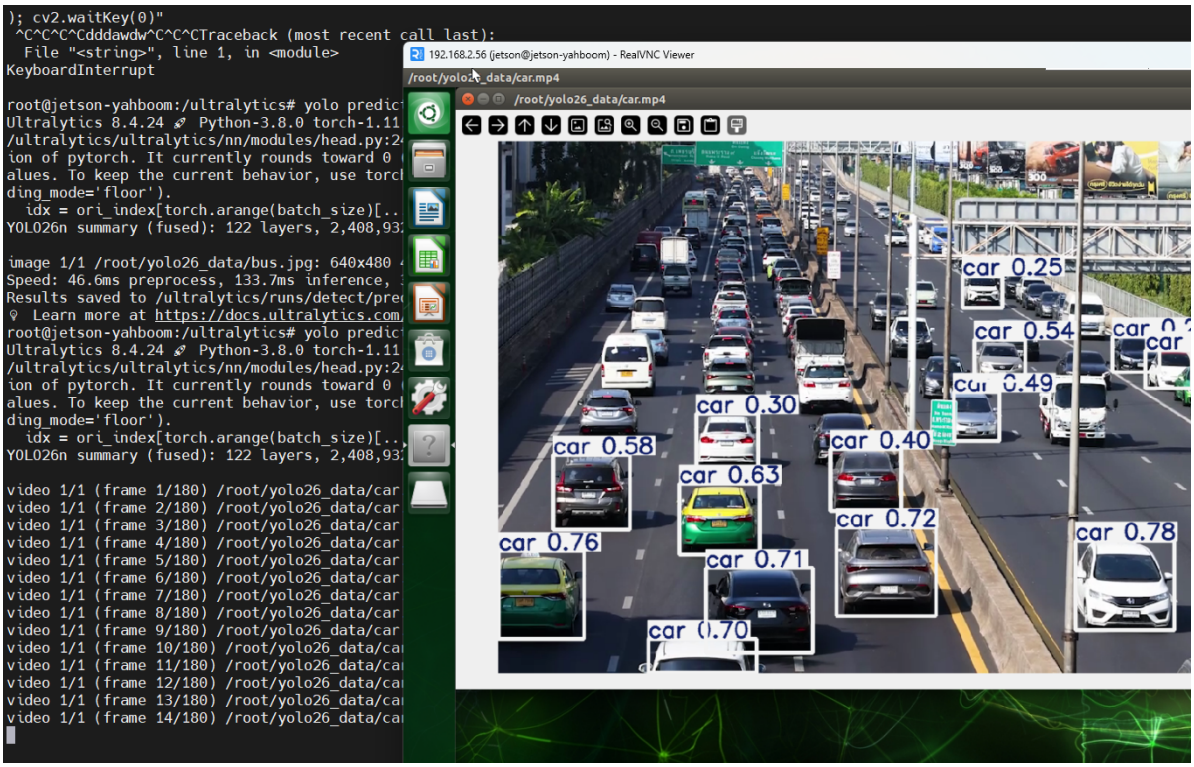


Then it can be downloaded to the computer for viewing



Or we can directly add the show parameter to watch the video in vnc

```
yolo predict model=yolo26n.pt source='/root/yolo26_data/car.mp4' show
```



5. Real-time Prediction

> Note: CLI-based real-time prediction is more suitable for USB cameras; for CSI cameras, it is recommended to use the corresponding `*_camera_csi.py` script in the `yahboom_demo` directory

Use yolo26n.pt to predict **USB camera** footage: If the system cannot find the corresponding model file in the directory where the command is run, it will automatically download (if download fails, you can copy the model in yourself)

Enter the project folder:

```
cd /ultralytics
```

Use yolo26n.pt to detect camera footage and output results: USB cameras can directly set `source` to the camera number

```
yolo predict model=yolo26n.pt source=0 save=False show # Object detection
# Instance segmentation: yolo predict model=yolo26n-seg.pt source=0 save=False
show
# Image classification: yolo predict model=yolo26n-cls.pt source=0 save=False
show
# Pose estimation: yolo predict model=yolo26n-pose.pt source=0 save=False show
# Oriented object detection: yolo predict model=yolo26n-obb.pt source=0
save=False show
```

Press "Ctrl + C" shortcut key in the terminal to terminate the program!

If using CSI camera, please enter the example code directory and run the corresponding Python script, for example:

```
cd /ultralytics/yahboom_demo
python3 01.detection_camera_csi.py
```

Scripts corresponding to other tasks are as follows:

```
python3 02.segmentation_camera_csi.py
python3 03.pose_camera_csi.py
python3 04.classification_camera_csi.py
python3 05.obb_camera_csi.py
```

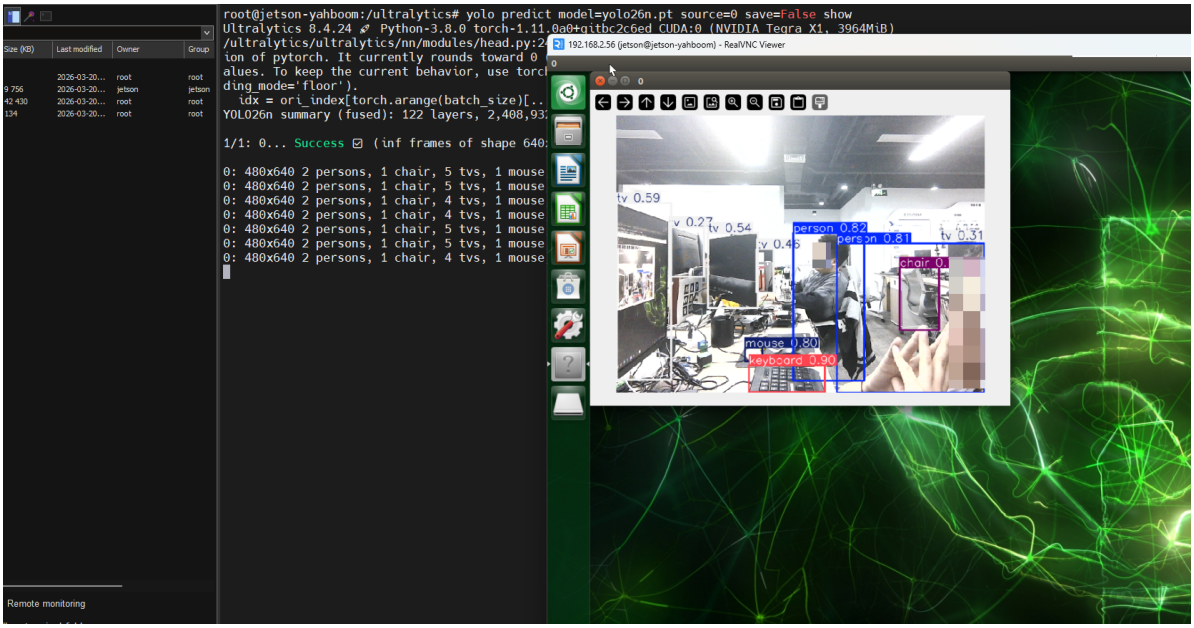
> **Parameter Description**

`model`: Specify YOLO model

`source`: Specify recognition source: multiple cameras can switch numbers

`save=False`: Disable saving results

`show`: Real-time display



References

Home - Ultralytics YOLO Documentation

Command Line Interface - Ultralytics YOLO Documentation